

01. VANTEC CORE L1: MOTOR (API & PERSISTENCIA)

Estatus: Fuente de Verdad Innegociable para Desarrollo de API y Persistencia. **Visión:** "Data-First" (Gobernanza de Base de Datos).

1. EL PRINCIPIO "DATA-FIRST" (GOBERNANZA DE DB)

En la ingeniería Vantec, la base de datos es el **plano arquitectónico** inmutable del sistema. El código debe adaptarse a la base de datos, y no al revés.

1.1 Arquitectura Dictada por Datos

- **Integridad Estructural:** Se prohíbe alterar tablas por "comodidad" del frontend. Se deben respetar las restricciones de integridad y normalización.
- **El Script Maestro (DDL):** El contenido de la Sección 4 de este manual es la ley. El Backend debe reflejar este esquema al 100% mediante modelos estrictamente tipados.

1.2 Rendimiento en el Motor (Server-Side Logic)

- **Agregaciones Masivas:** Todo cálculo de sumatorias o filtros complejos DEBE resolverse en SQL (Vistas Materializadas) para no saturar la memoria de la API.
-

2. CONTRATOS DE API Y PROGRAMACIÓN DEFENSIVA

La API funciona como el contrato legal entre los datos y la interfaz.

2.1 Inmutabilidad de Esquemas

- Todo Endpoint debe implementar esquemas de petición y respuesta estrictamente tipados. Si el contrato falla, la petición se rechaza en el borde (400 Bad Request).

2.2 Aislamiento Multi-Tenant (Default Deny)

- Ninguna consulta a la DB puede omitir el filtro de seguridad por `tenant_id` o `entidad_id`. El acceso está denegado por defecto.
-

3. RESILIENCIA Y MANEJO DE ESTADOS

- **Idempotencia:** Verificación obligatoria de existencia previa mediante UUID o Hash antes de cualquier inserción.
 - **Zero-Mock Policy:** Prohibido el uso de datos hardcodeados en el código de producción.
-

4. ESTRUCTURA DE PERSISTENCIA (DDL MAESTRO L6)

A continuación se detalla la estructura oficial de la bóveda atómica.

4.1 Extensiones y Seguridad

El motor requiere la generación nativa de UUIDs para garantizar la idempotencia global.

```
CREATE EXTENSION IF NOT EXISTS "uuid-ossf";
CREATE EXTENSION IF NOT EXISTS plpgsql;
```

4.2 Tablas de Identidad y Acceso

```
-- Gestión de Clientes (Multi-Tenancy)
CREATE TABLE IF NOT EXISTS public.tenants (
    tenant_id uuid NOT NULL PRIMARY KEY DEFAULT uuid_generate_v4(),
    rfc character varying(13) NOT NULL UNIQUE,
    business_name character varying(200) NOT NULL,
    is_active boolean DEFAULT true
);

-- Usuarios y Roles de Seguridad
CREATE TABLE IF NOT EXISTS public.users (
    user_id uuid NOT NULL PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id uuid REFERENCES public.tenants(tenant_id) ON DELETE CASCADE,
    username character varying(100) NOT NULL UNIQUE,
    password_hash character varying(255) NOT NULL,
    rol character varying(20) DEFAULT 'VISOR'
);
```

4.3 Bóveda de Comprobantes (UUID-Centric)

```
CREATE TABLE IF NOT EXISTS public.comprobantes (
    uuid uuid NOT NULL PRIMARY KEY,
    entidad_id uuid NOT NULL REFERENCES public.tenants(tenant_id) ON DELETE CASCADE,
    serie character varying(25),
    folio character varying(40),
    fecha_emision timestamp without time zone NOT NULL,
    total numeric(18,6) NOT NULL,
    xml_path character varying(1000) NOT NULL,
    pdf_path character varying(1000),
    orphan_payment boolean DEFAULT false -- Marca para procesos huérfanos
);

-- Relación Atómica de Pagos
CREATE TABLE IF NOT EXISTS public.cfdi_relacionados (
    id SERIAL PRIMARY KEY,
    cfdi_id uuid REFERENCES public.comprobantes(uuid) ON DELETE CASCADE,
    uuid_relacionado character varying(36) NOT NULL,
```

```
    monto_pagado numeric(18,6),
    saldo_insoluto numeric(18,6)
);
```

4.4 Lógica de Negocio en Motor (Triggers y Vistas)

Vista Materializada (Semáforo de Pagos): Resuelve el estatus de las facturas PPD de forma instantánea.

```
CREATE MATERIALIZED VIEW v_ppd_semaforo_status AS
SELECT
  uuid, total,
  COALESCE(SUM(monto_pagado), 0) as total_pagado,
  CASE
    WHEN orphan_payment = TRUE THEN 'AUSENTE'
    WHEN (total - COALESCE(SUM(monto_pagado), 0)) < 1.00 THEN 'PAGADO'
    ELSE 'PENDIENTE'
  END as estado_semaforo
FROM comprobantes
GROUP BY uuid, total, orphan_payment;
```

 CHECKLIST DE CALIDAD (CTO)

Requisito	Descripción	Estado
Integridad DDL	¿La DB refleja exactamente el script de la sección 4?	☒
Multi-Tenant	¿Todas las tablas críticas tienen filtro por <code>tenant_id</code> ?	☒
Idempotencia	¿Se usan <code>uuid</code> como llaves primarias en todo el sistema?	☒
Server-Side	¿Los cálculos de estatus (Semáforo) corren en SQL?	☒